

JiT 结构改进方案：面向 Flow Matching 的 FFN 与条件注入设计

整体判断

之前在 JiT 上做的六种 FFN 内部改造 (Timestep-Adaptive Gate、Flow-Evolved Gate、Spatial-Adaptive Value、Frequency-Split、Progressive Refinement、Multi-Step Integrated) 几乎全部与 baseline 重合，这个结果本身是有结论的：在 AdaLN-Zero 已经把 timestep 信息注入到每一层 LayerNorm 的 scale/shift/gate 的前提下，再在 SwiGLU 内部加第二次 timestep 门控，是信息角色重复、梯度路径冗余的。FFN 作为 channel mixer，它的 capacity 早已被 2 个 $4d \cdot d$ 的大矩阵吃满；在其内部再塞一个小 gate，对优化目标的改变量远低于噪声水平。

因此新一轮改造应当遵循两条原则：

1. 如果要碰 FFN，就把条件注入点移到 FFN "边界之外"——要么在 FFN 输入前，要么在 FFN 的输入端注入 AdaLN 做不到的信息（空间先验、频率分解），而不是再去重复 AdaLN 已经在做的 "global scale/shift"。
2. 如果要碰条件注入，就直接削减 AdaLN 本身——DiT/JiT 的 AdaLN-Zero 每 block 占 $6d^2$ 参数，在 JiT-XL 量级会占到总参数的 $\sim 27\%$ 。([arxiv](#)) ([Oxen](#)) FiTv2、PixArt- α 、DiffiT、U-ViT 都证明这部分有 2-14 $\times$ 的压缩空间，([The Moonlight +2](#)) 且不掉点甚至更好。对 flow matching 目标（时间 t 连续均匀、速度场光滑）尤其友好，因为调制信号不再需要那么高频地表达 t 。

以下六个方案即围绕这两条原则展开：方案 1-3 改造 FFN 的"边界与输入"，方案 4-6 替换 AdaLN 本身。所有方案都以 JiT-B/16 (pixel-space x-prediction, ImageNet 256, flow matching / v-loss 训练目标) 作为默认实验定位。

方案 1：Pre-Linear Timestep Gate SwiGLU

核心理想

不在 SwiGLU 内部加 gate (已证明无效)，而是在 token x 进入 FFN 的第一个 Linear 之前，用一个只依赖 timestep 的标量/向量 gate 筛一遍 x 。语义上这是在问："在当前时间 t ，哪些 channel 需要被 FFN 处理，哪些应该直接透过残差传下去？"

结构

记 block 输入 $x \in \mathbb{R}^{N \times d}$ ，AdaLN 调制后的输入 $\tilde{x} = \text{LN}(x) \cdot (1 + \gamma_2) + \beta_2$ ，timestep embedding $t_{\text{emb}} \in \mathbb{R}^{d_t}$ 。引入 pre-linear gate：

$$g(t) = \sigma(W_g t_{\text{emb}} + b_g) \in \mathbb{R}^d, \quad W_g \in \mathbb{R}^{d \times d_t}$$

$$\tilde{x}' = g(t) \odot \tilde{x}$$

$$\text{FFN}(\tilde{x}') = (\text{SiLU}(\tilde{x}'W_1) \odot (\tilde{x}'W_2)) W_3$$

其中 $g(t)$ 沿 sequence 维度广播，**不带 spatial 依赖**——这是与方案 2 的明确分工。关于"后面还要不要再 gate"，我们建议**不要**：后续的 α_2 残差门控（AdaLN-Zero 的 gate）已经覆盖了"整块 FFN 要不要生效"这件事，再加一个会重新陷入"和 AdaLN 角色重复"的老问题。

为什么这样设计成立

关键在于位置差异带来的信息角色差异：

- **AdaLN 的 γ, β** 是在 LayerNorm 之后做 "rescale + reshift"，本质是把零均值单位方差的激活重新"校准"回一个与 t 相关的分布。它调的是"什么分布"。
- **FFN 内部 gate**（之前失败的方案）是在 $\tilde{x}W_1$ 之后再 gate 一次 hidden activation。但 W_1 是一个线性变换，"先 gate $\tilde{x}$ 再过 W_1 " 和 "先过 W_1 再 gate $\tilde{x}W_1$ " 在 W_1 学到合适权重后几乎等价——这解释了为什么它无效：它是个 **可被 W_1 吸收的冗余自由度**。
- **Pre-linear gate $g(t) \odot \tilde{x}$** 发生在 $\tilde{x}$ 已经被 AdaLN 校准之后，但尚未被 FFN 的 W_1 线性展开之前。它 **不是 rescale 分布**（AdaLN 做了），也不是 rescale hidden（会被 W_1 吸收），而是 **在时间维上做 input-level selective pass**——挑选哪些 channel 的"已校准 token"被送进非线性 mixer。

这个角色在 flow matching 下尤其有用。flow matching 的速度场 $v_\theta(x_t, t)$ 在 $t \rightarrow 0$ 附近接近恒等拷贝（ $v \approx x_1 - x_0$ [\(arxiv\)](#) 的早期段由数据支配），在 $t \rightarrow 1$ 附近接近全局清理。一个 t -dependent input-mask 允许早期让更多 channel "skip" FFN（因为那些 channel 本质上应当直接透过），晚期让所有 channel 都进 FFN。

和已有工作的关系

- **Highway Networks / Gated Residual** (Srivastava 2015)：门控残差的元祖，gate 决定 "transform vs carry"。本方案是其 timestep-conditioned 版本。
- **Mixture-of-Depths** (Raposo 2024)：每 token 决定是否跳过一个 block。本方案是 per-channel 版，粒度更细，但只由 t 决定、不由 x 决定，优化更稳定。
- **AdaLN-Zero 的 α gate**： α_1, α_2 调的是整块 attention/FFN 的残差权重。 [\(APXML\)](#) 本方案的 $g(t)$ 调的是 FFN 输入的 per-channel 权重，粒度不同。

参数预算

$W_g \in \mathbb{R}^{d \times d_t}$ ，若 $d_t = d$ 则每 block d^2 参数（约为 AdaLN 的 1/6）。在 JiT-B ($d=768$) 每 block $\approx 0.59\text{M}$ ，12 层约 7M，占比 $<3\%$ 。可忽略。

风险 / 适合的实验定位

- **风险 1： σ 饱和**。如果 $g(t)$ 学到全部接近 1，方案退化为 identity。初始化建议 $b_g = 0$ 、 W_g 用 [xavier_uniform/10](#)，使初期 $g(t) \approx 0.5$ ，让 gate 有梯度。

- **风险 2：**和 AdaLN-Zero 的 α_2 叠加后"双重衰减"。由于 $g(t) \odot \tilde{x}$ 后又要乘 α_2 ，初始时整条 FFN 输出可能过小。建议 g 初期 bias 偏向 1 ($b_g = 2$ 使 $\sigma(2) \approx 0.88$)。

实验定位：JiT-B/16, ImageNet 256, flow matching velocity-prediction, baseline 400 epoch。记录 FID-50K、每 block gate 均值随 t 的曲线（用来验证 gate 确实在选择性工作）。如果 FID 打平但 gate 曲线呈"早期低、晚期高"的系统性趋势，说明机制生效但容量瓶颈不在这。

方案 2：Class-Conditioned Spatial Information Injection

核心理想

JiT 给模型的条件只有 (t, y) ，其中 y 是 class embedding——这两者都没有空间信息。模型全部的空间先验都来自 positional embedding。在 JiT 典型设定下（256² 图像，patch 16 → 只有 16×16 = 256 个 token，每个 patch 是 $16 \times 16 \times 3 = 768$ 维高维 pixel patch），(arxiv) token 数少而每个 token 维度高，(alphaXiv) spatial prior 不足很可能是 FID 瓶颈之一。

方案是：用一个很小的子网络从 class embedding y 学出一组 **per-spatial-position** 的条件 bias，作为额外信号注入 FFN 的 **value 分支**（SwiGLU 的 up-projection 分支）。这相当于把 "class → image" 的 spatial layout 先验显式地写进 FFN。

结构

记 class embedding $y \in \mathbb{R}^{d_y}$ ，token 数 $N = H \cdot W$ （JiT-B/16 时 $N = 256$ ）。定义一个共享的 **spatial bank** $B \in \mathbb{R}^{N \times r}$ ($r \ll d$ ，比如 $r = 16$)，和一个 **class-to-spatial 投影** $U \in \mathbb{R}^{r \times d}$ ：

$$s(y) = \tanh(W_y y) \in \mathbb{R}^r, \quad W_y \in \mathbb{R}^{r \times d_y}$$

$$S(y) = B \odot s(y)^\top U \in \mathbb{R}^{N \times d}$$

（这里 $B \odot s(y)^\top$ 是把 class-dependent 系数 $s(y)$ 与 spatial bank 的每一行做 Hadamard，再通过 U 升维到 d 。）

在每个 block 的 FFN 中注入到 SwiGLU 的 value 分支：

$$u = \tilde{x}W_1, \quad v = \tilde{x}W_2 + S(y)$$

$$\text{FFN}(\tilde{x}) = (\text{SiLU}(u) \odot v) W_3$$

关键约束： $S(y)$ 的计算 **跨 block 共享**（只算一次），只有 W_y 和 U 可以跨 block 不共享（更激进：甚至 U 也共享，每 block 只有一个 low-rank $V^{(l)} \in \mathbb{R}^{r \times r}$ 做 per-block 调整）。

为什么这样设计成立

1. **Class embedding 本身没有空间结构**：在 JiT 中 y 进入 AdaLN，经过 W_{adaLN} 变成 per-channel 的 γ, β ，这些对所有 N 个 token 一视同仁。模型从未从 class 得到过 "第 i 个位置应当是什么内容" 的 hint。
2. **少 token $\times$ 高维 patch** 是 JiT 的特殊设定：MAR/DiT 在 latent space 是 $32 \times 32 = 1024$ tokens + $d = 1152$ ，spatial redundancy 很高；JiT pixel space 是 $16 \times 16 = 256$ tokens，[arXiv](#) [arXiv](#) 每个 token 要独立承担更大的内容，spatial prior 不足会导致 "哪儿该有什么" 这个问题学得慢。
3. **注入 value 分支而非 gate 分支**：SwiGLU 中 $u = \text{SiLU}(\tilde{x}W_1)$ 负责 "selector"， $v = \tilde{x}W_2$ 负责 "content"。我们把 class-conditioned spatial bias 加到 **content** 上，相当于 "在 class y 下，位置 i 的默认内容是 $S(y)_i$ "，让 FFN 的 gate 选择器 u 去挑该不该保留这一份空间先验。这比加到 gate 分支更符合 inductive bias：spatial prior 是 **内容**，不是 **开关**。
4. **Spatial bank B 是可学习的 "位置 codebook"**：每一列 $B_{:,k}$ 代表一种可能的空间模式（类似 StyleGAN 的空间 noise）， $s(y)$ 决定这个 class 激活哪几种空间模式的混合。这比 "直接从 y 全连接到 $N \cdot d$ 维 per-spatial bias" 参数量低两个数量级。

和已有工作的关系

- **StyleGAN 的 spatial noise**：每 layer 注入空间无关的 per-pixel noise。本方案是 class-conditioned 版的 noise，但空间模式由学出来的 bank B 表达，而不是 Gaussian。
- **AdaIN with spatial extension / SPADE** (Park 2019)：用 semantic layout map 做 spatial affine。本方案等价于 "从 class 学出一张 layout"，但只在 FFN value 上加 bias、不 rescale。
- **PixArt- α 的 T5 cross-attention**：T2I 任务中 text 本身是 token 序列，通过 cross-attention 注入 spatial-aware 信号。[Superteams](#) 本方案把 **class (非序列)** 扩展成伪-spatial-token，然后通过直接加法而非 attention 注入，参数省 10 $\times$ 。
- **Register tokens** (Darcet 2024)：用少量 learnable tokens 吸收全局信息。[OpenReview](#) 本方案可以看作 "class 生成的 register"，但直接写到每个位置上而非作为可被 attention 的 token。

参数预算

- $B \in \mathbb{R}^{N \times r}: N \cdot r, N = 256, r = 16 \rightarrow 4\text{K}$ 。
- $W_y \in \mathbb{R}^{r \times d_y}: r \cdot d_y, d_y = d = 768 \rightarrow 12\text{K}$ 。
- $U \in \mathbb{R}^{r \times d}: r \cdot d \rightarrow 12\text{K}$ 。
- 总计 ~28K 参数，**全模型共享一份**。相比 JiT-B 总参数 130M，占比 <0.02%。

若每 block 独立 $W_y^{(l)}, U^{(l)}$ ，则乘以 $N_{\text{block}} = 12 \approx 290\text{K}$ ，仍可忽略。

风险 / 适合的实验定位

- **风险 1：unconditional 时 $S(y)$ 行为**。CFG 训练会有 $y = \emptyset$ 的情况，需要让 $S(\emptyset) = 0$ (用一个 learnable null embedding $y_\emptyset$ 并初始化为 0)。

- **风险 2：与 positional embedding 的冗余。**如果 B 学成了纯 positional (与 y 无关)，等价于 per-position learnable bias，无新信息。建议监控 $\|s(y) - \bar{s}\|$ (不同 class 的 spatial activation 差异)，若接近 0 说明机制未触发。
- **风险 3：patch 大 / token 少时 bank 容量有限。** $B \in \mathbb{R}^{256 \times 16}$ 的秩最多 16，可能不够表达 1000-class ImageNet。若失败，可考虑 $r \rightarrow 32$ 或把 B 做成两层 MLP ($y \rightarrow \mathbb{R}^{N \times d}$) 但参数更多。

实验定位：JiT-B/16 class-conditional ImageNet 256, flow matching + CFG=1.5。核心 metric 是 FID-50K；辅助 metric 是 **Inception Score** (spatial prior 如果生效应先体现在 IS 上而非 FID)。推荐先做 400 epoch 短跑验证方向，再扩到 1000 epoch 全跑。

方案 3：Input-Stage Frequency-Split FFN

核心理想

之前的 Frequency-Split 方案是在 FFN 内部把 hidden dim $4d$ 拆成两半、各处理一半，略差于 baseline——因为“让半条 hidden 去处理高频”相当于 **单纯削弱每支的 capacity**，并没有真正做频率分解。新方案是在 **FFN 输入前** 先用简单的 pooling + 上采样做低通滤波，让高低频在进入 FFN 时已经在空间上分开，两支 FFN 看到的是**真实的两种不同分布**。

结构

记 token 序列 $\tilde{x} \in \mathbb{R}^{N \times d}$ ，reshape 回 $\tilde{X} \in \mathbb{R}^{H \times W \times d}$ (JiT 中 $H = W = 16$)。

低频分支：在空间做 2×2 average pooling ($HW \rightarrow hw/4$)，再 nearest-neighbor 上采样回 $H \times W$ ：

$$\tilde{X}_{\text{low}} = \text{NN-up}(\text{AvgPool}_{2 \times 2}(\tilde{X})) \in \mathbb{R}^{H \times W \times d}$$

高频分支：

$$\tilde{X}_{\text{high}} = \tilde{X} - \tilde{X}_{\text{low}}$$

两条路径各过一个**窄 SwiGLU** (hidden dim = $2d$ 而非 $4d$ ，总参数持平)：

$$y_{\text{low}} = \text{SwiGLU}_{\text{low}}(\tilde{X}_{\text{low}}; \text{hidden} = 2d)$$

$$y_{\text{high}} = \text{SwiGLU}_{\text{high}}(\tilde{X}_{\text{high}}; \text{hidden} = 2d)$$

融合：由 timestep 控制低/高频权重：

$$(w_l, w_h) = \text{softmax}(W_f t_{\text{emb}}) \in \mathbb{R}^2$$

$$y = w_l(t) \cdot y_{\text{low}} + w_h(t) \cdot y_{\text{high}}$$

reshape 回序列。

参数共享优化：更省的变体是让两支共享 W_3 (down-projection)，仅 W_1, W_2 独立——此时两支共 $2 \cdot 2(d \cdot 2d) + d \cdot 2d = 10d^2$ ，与标准 SwiGLU 的 $3 \cdot 4d^2 = 12d^2$ 相当甚至更小。

为什么这样设计成立

1. ****与之前失败方案的关键区别****：之前是 "hidden space 的 channel split"，相当于把 W_1 的 $4d$ 维 output 划两组，每组看到完整输入但只用一半 channel——这不是频率分解，是容量削减。新方案在 ****空间维度**** 做真·低通滤波， $\tilde{X}_{\text{low}}$ 与 $\tilde{X}_{\text{high}}$ 在输入阶段就是两种不同的信号（分别 smooth 和 residual）。
2. **AvgPool + NN-up 是一个合理的 low-pass filter**：在频域上相当于一个 box filter，会去除 $> \pi/2$ 的空间频率。虽然不是理想 sinc，但足够把 "全局色调、轮廓" 和 "纹理、边缘" 分开。对 pixel-space generation，这两种信息学习 dynamics 不同——低频在 $t \rightarrow 1$ 早期就确定，高频在 $t \rightarrow 0$ 晚期才生成。
3. **Timestep-controlled fusion 匹配 diffusion 的 coarse-to-fine 本质**：多个工作（Rissanen 2022 blurring diffusion、Hooeboom 2022 cascaded diffusion）已证明 "早期偏低频、晚期偏高频" 是 diffusion 的内在 inductive bias。此处用 $w_l(t), w_h(t)$ 把这个先验显式编码进网络。
4. **高频分支残差结构可学习到 sharpness**： $\tilde{X} - \tilde{X}_{\text{low}}$ 包含了所有边缘和纹理。独立的 SwiGLU 让模型专门学习如何 denoise 高频（通常是最难的部分），避免和低频目标竞争 capacity。

和已有工作的关系

- **FreqFormer / FNet / Adaptive Frequency Filters** 等频域 Transformer：大多用 FFT 做频率分解。本方案用简单的空间 pool 做 proxy，计算量几乎为零（ 2×2 AvgPool + NN-up 在 CUDA 上是 memory-bound 常数时间）。
- **Blurring Diffusion / Cold Diffusion**（Rissanen 2022, Bansal 2022）：把 "blur" 作为 diffusion 过程本身的一部分。本方案不改 noise schedule，只是让网络内部感知频率分解。
- **U-Net 的多尺度结构**：U-Net 通过 downsample/upsample 天然做多频率处理；DiT/JiT 是 isotropic ViT 没有这个结构。本方案把 U-Net 的多尺度 inductive bias 压缩进单个 block 的 FFN，保留 ViT 的 isotropic 优势。
- **Swin V2 的局部窗口 attention**：在 attention 端做空间分解。本方案是 FFN 端的空间分解，互补。

参数预算

- 两支 SwiGLU (hidden=2d, 独立 W_1, W_2 , 共享 W_3) : $2(2 \cdot d \cdot 2d) + d \cdot 2d = 8d^2 + 2d^2 = 10d^2$ 。
- 原标准 SwiGLU (hidden=4d) : $3 \cdot 4d^2 = 12d^2$ 。
- 融合 $W_f \in \mathbb{R}^{2 \times d_t} : \approx 2d$, 可忽略。

结论：参数量略低于 baseline SwiGLU (~17% 省), 计算量也略低 (两支 2d hidden 的 GeLU/SiLU 总 FLOPs $\approx$ 原 4d 的 ~67%)。

风险 / 适合的实验定位

- 风险 1：AvgPool 边界效应。** 16×16 网格做 2×2 pool 没问题, 若 patch 数不是 2 的倍数需要 padding。JiT-B/16 at 256² (16×16) 和 JiT-XL/32 at 512² (16×16) 都满足。
- 风险 2：NN-up 引入的 block artifact。** 高频分支 $\tilde{X} - \tilde{X}_{\text{low}}$ 会在 2×2 block 边界有人为的阶跃, 模型可能学到这个 artifact 作为 shortcut。可用 bilinear-up 替代 NN-up, 但计算略贵。
- 风险 3：和 attention 的重复。** Attention 已经做了全局低频的 token mixing。若 FFN 再做低频, 可能冗余。建议 ablation：仅高频分支 vs 仅低频分支 vs 融合。
- 风险 4：pixel-space vs latent-space 差异。** JiT 是 pixel-space, 高频信号强且丰富, 本方案预期收益高；若迁移到 latent-space (SiT/DiT), latent 已经是低频表征, 高频分支可能无信号。

实验定位：JiT-B/16 pixel-space ImageNet 256, v-prediction flow matching。主 metric FID-50K。推荐消融：(a) 固定 $w_l = w_h = 0.5$ ；(b) 去掉高频分支 (纯低频)；(c) 去掉低频分支 (纯高频)；(d) 完整方案。如果 (d) $\gg$ (a)~(c), 证明分频+imestep 融合是核心贡献。

方案 4：AdaLN-LoRA (FiTv2 路线)

核心理想

DiT 的 AdaLN-Zero 每 block 生成 6 个调制信号, 用一个独立的 $W^{(l)} \in \mathbb{R}^{6d \times d}$, 总参数 $N_{\text{block}} \cdot 6d^2$ 。FiTv2 (Wang et al. 2024, arxiv 2410.13925) 的核心观察是：所有 block 生成调制信号时, 绝大部分权重可以共享——block 之间的差异用一个低秩增量表达即可。

结构

所有 block 共享一个全局 AdaLN 投影 $W^g \in \mathbb{R}^{6d \times d}$, 每 block 只保留一个 rank- r 的 LoRA 增量：

$$S^{(l)}(c) = W^g c + W_2^{(l)} W_1^{(l)} c$$

其中 $c = \text{MLP}(t_{\text{emb}} + y_{\text{emb}})$, $W_1^{(l)} \in \mathbb{R}^{r \times d}$, $W_2^{(l)} \in \mathbb{R}^{6d \times r}$ 。

$$[\gamma_1^{(l)}, \beta_1^{(l)}, \alpha_1^{(l)}, \gamma_2^{(l)}, \beta_2^{(l)}, \alpha_2^{(l)}] = \text{chunk}(S^{(l)}(c), 6)$$

其余 AdaLN-Zero 用法不变 (α 零初始化, γ, β 用于 $\text{LN}(x)(1 + \gamma) + \beta$)。

python

```
class AdaLNLoRA(nn.Module):
    def __init__(self, d, num_blocks, rank=None):
        super().__init__()
        rank = rank or d // 16
        self.W_global = nn.Linear(d, 6 * d, bias=True) # shared
        self.W1 = nn.Parameter(torch.zeros(num_blocks, rank, d))
        self.W2 = nn.Parameter(torch.zeros(num_blocks, 6 * d, rank))
        nn.init.kaiming_uniform_(self.W1, a=math.sqrt(5))
        # W2 保持 0 初始化 → LoRA 项初期为 0, 退化为 shared AdaLN

    def forward(self, c, block_idx):
        s = self.W_global(c) + (self.W2[block_idx] @ (self.W1[block_idx] @ c.T)).T
        return s.chunk(6, dim=-1)
```

默认 $\text{rank } r = d/16$ (FiTv2 默认值, JiT-B $d=768$ 时 $r = 48$)。

为什么这样设计成立

1. **调制信号的 block-wise 变化是低秩的**：FiTv2 文章通过实验发现, 不同 block 的 adaLN 输出权重矩阵 $W^{(l)}$ 之间的差异矩阵 $W^{(l)} - W^{(l')}$ 的奇异值谱很快衰减——这意味着 block 之间真正"不同"的部分可以用低秩捕获。
2. **zero-init 安全性**： W_2 零初始化 → LoRA 项为 0 → 每 block 初始等于 shared AdaLN → 对优化动态无扰动, 和 AdaLN-Zero 的"初期恒等"哲学一致。
3. **对 flow matching 特别友好**：flow matching 的 $v(x_t, t)$ 在 t 上更光滑 (没有 DDPM 的 σ_t 指数变化), 每 block 需要的 t -dependent 调制差异更小 → 低秩近似误差更小。

和已有工作的关系

- **FiTv2** (Wang et al. 2024, arxiv 2410.13925)：原始方法, FiTv2-XL 在 ImageNet 256 达 FID 2.15 (优于 DiT-XL 的 2.27), 收敛速度 2×。
- **LoRA** (Hu et al. 2021)：低秩适配的通用框架, 原用于 LLM 微调。这里是 **训练时就使用 LoRA 结构**, 而非微调。
- **PixArt-α 的 AdaLN-single** (方案 5)：另一种降参方式, 用可学习的 per-block offset 代替 LoRA。

参数预算

- Shared $W^g: 6d^2$, 一次。
- Per-block LoRA: $(d + 6d) \cdot r = 7dr$ 。
- 总: $6d^2 + N \cdot 7dr$ vs 原 $N \cdot 6d^2$ 。

JiT-B ($d=768, N=12, r=48$):

- 原 AdaLN: $12 \cdot 6 \cdot 768^2 \approx 42.5M$
- AdaLN-LoRA: $6 \cdot 768^2 + 12 \cdot 7 \cdot 768 \cdot 48 \approx 3.5M + 3.1M = 6.6M$
- 减少约 **6.4×** (JiT-XL $d=1152$ 量级可达 13-14×)。

节省下的 ~36M 参数建议重新投资到 **增加 block 数** (FiTv2 的做法) 或 **加宽 FFN hidden**, 保持总参数预算不变而获得 backbone 容量。

风险 / 适合的实验定位

- **风险 1: shared W^g 成为瓶颈**。如果 t, y 的表达需要在不同 block 截然不同的调制 (例如早几层做粗结构、后几层做细节), shared W^g 可能不够。消融: 对 r 做 {8, 16, 32, 64} 扫描。
- **风险 2: CFG 行为变化**。CFG 下 $y = \emptyset$ 的 branch 经过 shared W^g 可能不再严格正交于 conditional branch。建议沿用 FiTv2 的 null class 处理。

实验定位: 这是优先级最高的方案, 因为收益明确 (参数省 6×, FiTv2 已证明不掉点)。JiT-B/16 baseline $\rightarrow$ JiT-B/16 + AdaLN-LoRA, 保持总参数, 把省下的参数加到深度 (12 $\rightarrow$ 16 层) 或宽度 ($d=768 \rightarrow 896$)。目标: 相同参数下 FID 下降 ≥ 0.5 , 或相同 FID 下 params 降低 20%。

方案 5: AdaLN-Single with Per-Block Residual (PixArt- α 路线)

核心想法

PixArt- α 观察到 AdaLN 在 T2I 场景下占 DiT-XL 总参数的 27%, 极度冗余。他们的方案比 LoRA 更激进: 只用一个全局 MLP 处理 timestep, 每 block 只保留一个可学习的 per-block offset embedding (完全不依赖 c)。本方案将其适配到 JiT 的 class-conditional 设定。

结构

全局 AdaLN (跨所有 block 共享):

$$\bar{S}(c) = \text{MLP}_{\text{global}}(c) \in \mathbb{R}^{6d}$$

每 block 的调制 = 全局输出 + 可学习 per-block offset:

$$S^{(l)} = \bar{S}(c) + E^{(l)}, \quad E^{(l)} \in \mathbb{R}^{6d} \text{ (learnable parameter)}$$

$E^{(l)}$ 是纯参数，与 c 无关，仅 $6d$ 个标量每 block。初始化为 0 使初期所有 block 等价于 shared AdaLN。

为什么这样设计成立

1. c 的信息相对低：在 JiT 中 $c = f(t, y)$ 是一个 d 维向量，其在不同 block 的表达需要的差异有限——不同 block 的“任务差异”更多体现在静态 bias（早期层关注粗结构，晚期层关注细节），这部分可以用 per-block offset 承载；而“随 c 变化”的部分是共性，可以用 shared MLP 承载。
2. 比 LoRA 更省：LoRA 需要 $7dr$ per-block，此处只需 $6d$ per-block，再少一个数量级。
3. 与 AdaLN-Zero 初始化等价： $E^{(l)} = 0$ 时 block 间调制一致， α 零初始化 $\rightarrow$ 初期每 block 恒等，优化稳定。

和已有工作的关系

- **PixArt- α** (Chen et al. 2023, arxiv 2310.00426)：提出 adaLN-single，在 T2I 中 adaLN 参数占比从 27% 降到 $<2\%$ ，zero-shot COCO FID 7.32。
- **adaLN-single 的局限**：在 **class-conditional** ImageNet 上是否依然足够，尚无公开 ablation。PixArt- α 的成功部分依赖 T2I 还有 cross-attention 独立承载文本条件；JiT 是纯 class-conditional， c 只走 AdaLN 这条路， $E^{(l)}$ 能否独立承载 **block-wise** 差异有更高要求。因此本方案相对 LoRA（方案 4）更激进、风险更高。

参数预算

- Global MLP: $d^2 + 6d^2 = 7d^2$ （一次）。
- Per-block offset $E^{(l)}$: $6d \times N$ 。

JiT-B ($d=768, N=12$): $7 \cdot 768^2 + 12 \cdot 6 \cdot 768 \approx 4.1M + 55K \approx 4.2M$ 。比 LoRA 再省 $\sim 37\%$ ，比 baseline 省 **10 $\times$** 。

风险 / 适合的实验定位

- **风险 1：per-block 调制不足**。JiT 中 c 同时含 t 和 y ，block-wise 的“如何处理不同 class”可能需要更多 capacity，而 $E^{(l)}$ 不依赖 y ，表达力可能受限。
- **风险 2： γ, α 的 bias-only 调制**。per-block offset 只能加 bias，不能 rescale——如果某 block 需要对 shared $\bar{S}$ 做乘性调整（“这层对 timestep 更敏感”），本方案做不到。
- **缓解**：可以做混合方案—— γ, β 用 AdaLN-LoRA（rank- r ，per-block）， α 用 AdaLN-single（per-block offset 够用）。因为 gate α 的 block-wise 差异最小。

实验定位：作为 **AdaLN-LoRA** 的进一步消融。先确认方案 4 有效，再用方案 5 测是否能进一步减参。如果方案 4 打平 baseline 而方案 5 FID $+0.3 \sim +0.5$ ，说明 class-conditional 下 LoRA 是更好的甜

蜜点；如果方案 5 也打平，说明 JiT 的 AdaLN 冗余度极高，可以激进压缩。

方案 6：Time-Dependent QKV Projection (DiffiT 路线)

核心理念

AdaLN 的一个根本设计选择是 "条件调制 LayerNorm"——但 timestep 的信息其实更自然地融入到 attention 的 Q/K/V 投影中，直接影响 token 间的 similarity 计算。DiffiT (Hatamizadeh et al. 2024, arxiv 2312.02139) 提出 **Time-dependent Multi-head Self-Attention (TMSA)**：让 Q/K/V 的投影本身依赖 timestep，彻底跳过 AdaLN。

结构

标准 attention： $q = xW_q, k = xW_k, v = xW_v$ 。TMSA 额外让 timestep token $x_t \in \mathbb{R}^d$ (从 t_{emb} + y_{emb} 得到) 参与 projection：

$$q = xW_q^{(s)} + x_tW_q^{(t)}, \quad k = xW_k^{(s)} + x_tW_k^{(t)}, \quad v = xW_v^{(s)} + x_tW_v^{(t)}$$

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + B\right)V$$

$x_tW^{(t)}$ 沿 sequence 维度广播 (所有 token 加同一个 timestep-dependent bias)。可以理解为：在 t -dependent 的 "时间原点" 上做 attention。

FFN 侧不做调制 (或保留一个最小的 shift-only 调制)，完全去掉 AdaLN 的 γ, β, α 。RMSNorm 替代 LayerNorm 进一步省参数。

python

```

class TMSA(nn.Module):
    def __init__(self, d, n_heads):
        super().__init__()
        self.qkv_s = nn.Linear(d, 3 * d, bias=False)      # spatial QKV
        self.qkv_t = nn.Linear(d, 3 * d, bias=False)      # temporal QKV
        self.proj = nn.Linear(d, d)
        self.n_heads = n_heads

    def forward(self, x, x_t):
        # x: (B, N, d), x_t: (B, d) (broadcast to all tokens)
        qkv = self.qkv_s(x) + self.qkv_t(x_t).unsqueeze(1)
        q, k, v = rearrange(qkv, 'b n (three h d) -> three b h n d',
                             three=3, h=self.n_heads).unbind(0)
        attn = F.scaled_dot_product_attention(q, k, v)
        return self.proj(rearrange(attn, 'b h n d -> b n (h d)'))

```

为什么这样设计成立

1. **条件信息更直接进入 similarity**：AdaLN 只调 LayerNorm 的 scale/shift，time 对 attention 的影响需要经过一次 LayerNorm 再经过 QKV 投影——路径长、梯度弱。TMSA 让 time 直接决定 Q, K，**attention pattern 本身随 t 变化**，这对扩散模型至关重要（早期 t 需要全局 attention 建构图，晚期 t 需要局部 attention 修细节）。
2. **参数量减半**：每 block 从 AdaLN 的 $6d^2$ 降到 TMSA 的 $3d^2$ （三个额外的 temporal projections）。
3. **无需 zero-init 技巧**：AdaLN-Zero 需要 α 零初始化保证初期恒等；TMSA 中 $W^{(t)}$ 可以用标准 Xavier 初始化且不影响收敛，因为 x_t 本身是一个 learned embedding，从零 embedding 开始自然使 temporal bias 初期很小。

和已有工作的关系

- **DiffiT** (Hatamizadeh et al. 2024, arxiv 2312.02139)：原始方法。Latent DiffiT 在 ImageNet 256 达 FID 1.73（发布时 SOTA），每 block 参数省一半。
- **MMDiT / SD3** (Esser 2024)：用**双流 QKV**（image stream + text stream）后 joint attention，与 TMSA 类似但更复杂（为 T2I 设计）。
- **U-ViT** (Bao 2022)：把 time 作为额外 token 拼到 sequence 前，让 attention 自然注入。TMSA 是 U-ViT 的“权重分解版”——time token 不进 sequence（省 N 维 attention），但直接影响 QKV。
- 可与 **QK-Norm** (Dehghani 2023) 叠加稳定训练。

参数预算

- 每 block TMSA temporal projections: $3d^2$ 。

- 去掉 AdaLN: 省 $6d^2$ 。
- 净省： $3d^2$ per block。

JiT-B ($d=768, N=12$): 净省 $12 \cdot 3 \cdot 768^2 \approx 21M$ (约占 JiT-B 总参数 16%)。

RMSNorm 替代 LayerNorm 再省 $2d \cdot 2N = 4dN$ (忽略)。

风险 / 适合的实验定位

- **风险 1：FFN 无条件信号。** 如果完全移除 AdaLN, FFN 完全不知道 t 。DiffiT 论文保留了 FFN 前的 time-aware shift (bias-only), 建议沿用。可以加一个 per-block 的 $\beta^{(l)} = W_{\beta}^{(l)} c$ (仅 shift, 不 scale), 参数 d^2 per block, 仍比 AdaLN 省 5×。
- **风险 2：训练稳定性。** TMSA 没有 α zero-init, 若初期 temporal projection 输出过大, attention 可能被 time 主导。DiffiT 用了 learning rate warmup + QK-Norm 缓解, JiT 应当沿用。
- **风险 3：pixel-space 的 Q/K norm 不稳。** JiT pixel-space 激活方差大, QK-Norm 更必要, 否则 attention softmax 易饱和。

实验定位：作为最激进的 AdaLN 替代方案。与方案 4 (AdaLN-LoRA) 互补——前者是"小改 AdaLN", 后者是"换 attention"。建议在方案 4 验证有效后再试方案 6, 以获得" $6d^2 \rightarrow 3d^2 + \text{FFN-shift}(d^2)$ "的更大压缩。目标 FID 持平或 -0.2。

方案总结与推荐实验顺序

一句话总结

#	方案	一句话
1	Pre-Linear Timestep Gate	把 timestep gate 从 SwiGLU 内部移到 FFN 输入前, 与 AdaLN 的 LN-rescale 角色互补
2	Class-Conditioned Spatial Injection	用 low-rank spatial bank 从 class embedding 生成 per-position bias, 注入 FFN value
3	Input-Stage Frequency-Split FFN	在 FFN 输入前用 pool+upsample 分出高低频, 两支窄 SwiGLU 处理后 timestep-gated 融合
4	AdaLN-LoRA (FiTv2)	全局 shared AdaLN 投影 + per-block rank- r LoRA 增量, 参数省 6-14×
5	AdaLN-Single (PixArt- α)	全局 AdaLN + per-block 可学习 offset, 比 LoRA 再少 40% 参数
6	TMSA (DiffiT)	让 timestep 进入 Q/K/V 投影, 彻底去掉 AdaLN, 省 $3d^2$ per block

推荐实验顺序

优先级 1（高置信度、高收益）：方案 4 **AdaLN-LoRA**。FiTv2 已有公开的 FID 2.15 数据（优于 DiT-XL 2.27），迁移到 JiT 的风险极低。预期收益：参数省 6×，可把省下的参数投入深度/宽度，FID 下降 0.3-0.5。

优先级 2（高置信度、中收益）：方案 3 **Input-Stage Frequency-Split FFN**。机制清晰、参数不增、计算略降，且 pixel-space 的 JiT 对频率分解特别契合（latent space 已是低频表征、pixel space 保留完整频谱）。预期收益：FID 打平到 -0.3。

优先级 3（中置信度、可能大收益）：方案 2 **Class-Conditioned Spatial Injection**。针对 JiT 少 token × 高维 patch 的 bottleneck，注入 spatial prior 有明确理论动机；但参数极省（<0.1M），值得一试。预期收益：IS 提升显著，FID 可能 -0.2~-0.5。

优先级 4（探索性）：方案 6 **TMSA**。激进替换 AdaLN，需要配合 QK-Norm 和 warmup；若成功可把 AdaLN 全部移除。建议在方案 4 成功后作为"是否能更激进"的探索。

优先级 5（消融/对比）：方案 1 **Pre-Linear Gate** 和方案 5 **AdaLN-Single**。作为方案 4/6 的消融对照——前者验证"FFN 边界 gate 的意义"，后者验证"JiT 的 AdaLN 能否极端压缩"。

最终建议的组合方案

若只做一个实验：**方案 4 (AdaLN-LoRA, $r=d/16$) + 方案 3 (Input-Frequency-Split FFN)**。两者正交（一个改条件注入、一个改 FFN），参数总体持平或略降，机制互补（前者释放 AdaLN 冗余容量，后者强化 FFN 频率感知）。这组合最有可能在相同参数预算下给出 FID 明显下降的结果，且风险最低。

参考文献

1. Peebles, W., & Xie, S. (2023). *Scalable Diffusion Models with Transformers*. arXiv:2212.09748. (DiT, AdaLN-Zero)
2. Li, T., & He, K. (2025). *Back to Basics: Let Denoising Generative Models Denoise*. arXiv:2511.13720. (JiT)
3. Ma, N., et al. (2024). *SiT: Exploring Flow and Diffusion-based Generative Models with Scalable Interpolant Transformers*. arXiv:2401.08740.
4. Wang, Z., et al. (2024). *FiTv2: Scalable and Improved Flexible Vision Transformer for Diffusion Model*. arXiv:2410.13925. (AdaLN-LoRA)
5. Chen, J., et al. (2023). *PixArt- α : Fast Training of Diffusion Transformer for Photorealistic Text-to-Image Synthesis*. arXiv:2310.00426. (AdaLN-single)
6. Hatamizadeh, A., et al. (2024). *DiffiT: Diffusion Vision Transformers for Image Generation*. arXiv:2312.02139. (TMSA)
7. Bao, F., et al. (2023). *All are Worth Words: A ViT Backbone for Diffusion Models*. arXiv:2209.12152. (U-ViT)

8. Esser, P., et al. (2024). *Scaling Rectified Flow Transformers for High-Resolution Image Synthesis*. arXiv:2403.03206. (MMDiT / SD3)
9. Perez, E., et al. (2018). *FiLM: Visual Reasoning with a General Conditioning Layer*. AAAI. arXiv:1709.07871.
10. Yao, J., et al. (2025). *Reconstruction vs. Generation: Taming Optimization Dilemma in Latent Diffusion Models*. arXiv:2501.01423. (LightningDiT / VA-VAE)
11. Hu, E., et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685.
12. Darcet, T., et al. (2024). *Vision Transformers Need Registers*. ICLR. arXiv:2309.16588.
13. Park, T., et al. (2019). *Semantic Image Synthesis with Spatially-Adaptive Normalization*. CVPR. (SPADE)
14. Rissanen, S., et al. (2022). *Generative Modelling with Inverse Heat Dissipation*. arXiv:2206.13397. (Blurring Diffusion)
15. Dehghani, M., et al. (2023). *Scaling Vision Transformers to 22 Billion Parameters*. ICML. (QK-Norm)
16. Raposo, D., et al. (2024). *Mixture-of-Depths*. arXiv:2404.02258.